Exercise Session 7

Systems Programming and Computer Architecture

Fall Semester 2026

Agenda

Systems@ETH Zürich

- Assignment 4 – Questions
- x86 Assembly Refresher Kahoot
- Alignment of Structs & Unions
- How to deal with binaries?
- GDB Demo – bomb defusal example
- Assignment 5 – Intro & Tips

Assignment 4 – Questions?



*This joke is written in Intel Syntax.

x86 Assembly Refresher Kahoot

https://create.kahoot.it/details/b03429fd-3368-4e16-b27c-347ef9f6ae16

Systems@ETH Zürich

Alignment of Structs & Union

Recap and Exam Questions

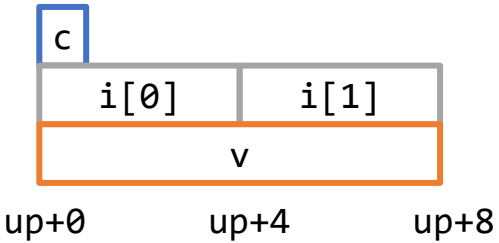
Alignment of Structs & Unions

Systems@ETH Zürich

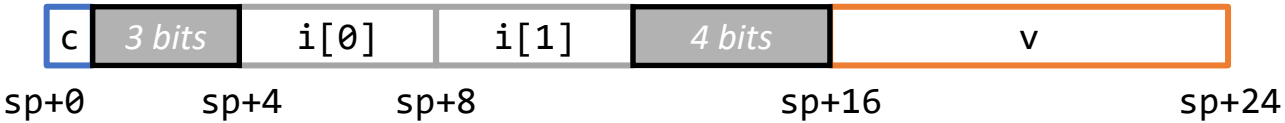
- A memory address is *K*-byte aligned if it's divisible by *K* (i.e. lowest $\log_2 K$ bits are `0` in binary representation)
- "natural" alignment: Primitive data types of *K* bytes are *K*-byte aligned.
- Structs:
  - Bytes are in order of declared fields
  - Padding added such that every element's alignment fulfilled
  - Struct *K*-byte aligned for largest element's alignment *K*
  - Padding at the end of struct added such that it's length is divisible by *K*
- Unions: aligned according to largest element's alignment

Alignment Example

```
union U1 {
  char c;
  int i[2];
  double v;
} *up;
```




```
struct S1 {
  char c;
  int i[2];
  double v;
} *sp;
```



Exam Question: HS16, Q6

```
struct rec {
    unsigned int id;
    char flag;
    short count;
    char valid;
};
```

Assuming standard alignment rules for x86-64, what is the value of:

`sizeof(struct rec)`

12

Exam Question: HS18, Q3

```
struct membuf {
    char valid;
    size_t length;
    void *buffer;
    unsigned int flags;
};
```

Assuming standard alignment rules for x86-64, what is the value of:

`sizeof(struct membuf)`

32

Exam Question HS19, Q4

```c
#include <stdio.h>
#include <stdint.h>

struct s1 {
    char f1;
    uint32_t f2;
};
union u1 {
    uint32_t f1;
    char f2;
    union u1 *f3;
};

int main() {
    char* p1 = (char*)(0x8000);
    printf("p1: %p\n", p1);
}
```

When compiled and run, the program outputs `p1 = 0x8000`.

For each of the following code fragments, say what the output will be when the `printf` statement in `main` is replace with that fragment.

Exam Question HS19, Q4

```c
#include <stdio.h>
#include <stdint.h>

struct s1 {
    char f1;
    uint32_t f2;
};
union u1 {
    uint32_t f1;
    char f2;
    union u1 *f3;
};

int main() {
    char* p1 = (char*)(0x8000);
    printf("p1: %p\n", p1);
}
```

When compiled and run, the program outputs `p1 = 0x8000`.

```c
struct s1 *p6 =
    ((struct s1 *)p1) + 5;
printf("p6 = %p\n", p6);
```

```
p6 = 0x8028
```

Exam Question HS19, Q4

```c
#include <stdio.h>
#include <stdint.h>

struct s1 {
    char f1;
    uint32_t f2;
};
union u1 {
    uint32_t f1;
    char f2;
    union u1 *f3;
};

int main() {
    char* p1 = (char*)(0x8000);
    printf("p1: %p\n", p1);
}
```

When compiled and run, the program outputs `p1 = 0x8000`.

```c
union u1 *p7 = ((union u1 *)p1) + 2;
printf("p7 = %p\n", p7);
```

`p6 = 0x8010`

Exam Question HS19, Q4

```c
#include <stdio.h>
#include <stdint.h>

struct s1 {
    char f1;
    uint32_t f2;
};
union u1 {
    uint32_t f1;
    char f2;
    union u1 *f3;
};

int main() {
    char* p1 = (char*)(0x8000);
    printf("p1: %p\n", p1);
}
```

When compiled and run, the program outputs `p1 = 0x8000`.

```c
struct s1 *a1 = (struct s1 *)p1;
printf("&a1[1] = %p\n", &a1[1]);
```

`&a1[1] = 0x8008`

Exam Question HS19, Q4

```c
#include <stdio.h>
#include <stdint.h>

struct s1 {
    char f1;
    uint32_t f2;
};
union u1 {
    uint32_t f1;
    char f2;
    union u1 *f3;
};

int main() {
    char* p1 = (char*)(0x8000);
    printf("p1: %p\n", p1);
}
```

When compiled and run, the program outputs `p1 = 0x8000`.

```c
struct s1 *a2 = (struct s1 *)p1;
printf(
    "&(a2->f2) = %p\n", &(a2->f2)
);
```

`&(a2->f2) = 0x8004`

Exam Question HS19, Q4

```c
#include <stdio.h>
#include <stdint.h>

struct s1 {
    char f1;
    uint32_t f2;
};
union u1 {
    uint32_t f1;
    char f2;
    union u1 *f3;
};

int main() {
    char* p1 = (char*)(0x8000);
    printf("p1: %p\n", p1);
}
```

When compiled and run, the program outputs `p1 = 0x8000`.

```c
struct s1 a3[5] = (struct s1 *)p1;
printf("&a3[1] = %p\n", &a3[1]);
```

Compile-time error

```
error: invalid initializer
```

Systems@ETH Zürich

Dealing with Binaries

Overview

Debugging



“If debugging is the process of removing bugs, then programming must be the process of putting them in.”

Motivation – Debugging opaque errors

Systems@ETH Zürich

C Source

```
int foo(char *a) {
    return strlen(a);
}

int main() {
   char *a = NULL;
   printf("%d", foo(a));
   return 0;
}
```

Output

Segmentation fault

Problem:

The output does not tell you where the Segmentation fault happened

Motivation – Security (see assignment)

Systems@ETH Zürich

- Exploits usually need precise knowledge of memory layout
  → not known prior to compilation
- Easier to track relevant information at runtime (stack evolution, register values)
- Defenses implemented in modern compilers:
  - Need to understand why they exist to (ab)use them

Getting the Assembly

`objdump:` display info about object files

`-d`	Display the assembly of the machine instructions (only sections which are expected to contain instructions)
`-D`	Display the assembly of all sections
`-l`	Display line numbers when debugging information are present
`-r`	Print the relocation entries
`-S`	Display the source code (only if possible)
`-t`	Display the symbol table entries
`-x`	Equivalent to `-a -f -h –p –r –t`

Getting String Info

`strings`

Prints printable character sequences of >3 chars with `'\0'` termination.

This is helpful to get strings used in the `printfs`

```c
#include <stdio.h>
int main() {
    printf("Result %d\n", 42);
}
```

```
> gcc main.c && strings a.out
/lib64/ld-linux-x86-64.so.2
__libc_start_main
…
u+UH
Result %d
9*3$"
…
```

`strings:` print the sequences of printable characters in files

`-a` Scan whole file, not just initialized and loaded sections

`-n` Change minimum string length

So Far



- Outputs show program structure
- but no information about execution

→ next step: run program in gdb

Solution

Systems@ETH Zürich

- Use a debugger to execute the program step by step
- In our case this will be gdb
  - With help from the binutils

GDB: Interactive Shell

gdb behaves pretty much like Linux shell

(auto completion, history of commands, …)

For Apple Silicon Users: Check out special setup on Moodle

- Online documentation
- Cheat Sheet
- GDB help
  `(gdb) help [command]`

Start GDB

Start GDB with binary as argument:

```
> gdb the_program
> (gdb)
```

Start GDB and then load binary afterwards:

```
> gdb
> (gdb) file the_program
```

`(gdb)` at begging of line indicates GDB is running

Run Program

Systems@ETH Zürich

- run program (also restart):
  `> (gdb) run`
- Program runs like in shell directly
- Additional information missing:
  - Function
  - Line
  - File
  - Location of crash

Breakpoint: Set

In file `file.c` at line 123:

```
> (gdb) break file.c:123
```

At function foo():

```
> (gdb) break foo
```

At address:

```
> (gdb) break *0x80487dd
```

Breakpoint at next instruction to be executed:

```
> (gdb) break
```

Delete breakpoint:

```
> (gdb) delete <breakpoint>
```

Show information about all breakpoints:

```
> (gdb) info breakpoints
```

Breakpoint: Execute

Systems@ETH Zürich

Until next source code line
(first instruction of new line)
(if debug info available)

```
> (gdb) step [n]
```

Until next line of source code but function calls are one instruction. (like step)

```
> (gdb) next [n]
```

One assembly instruction

```
> (gdb) stepi [n]
```

One instruction, but function calls are one instruction

```
> (gdb) nexti [n]
```

step goes **into** function calls,
next goes **over** them

Breakpoint: Execute

Systems@ETH Zürich

Until next breakpoint:

```
> (gdb) continue
```

Until current function returns

```
> (gdb) finish
```

Every time we hit a breakpoint:
The program pauses and gdb prompts for a command

Breakpoint: Conditional

Systems@ETH Zürich

Trigger breakpoint only if condition holds:

```
> (gdb) break file.c:123 if variable > 456
```

Also works for watchpoints

Watchpoint

- Set a watchpoint:
  `> (gdb) watch <variable>`
- Be informed about changes to a variable
- Like setting a breakpoint on the assignment operator for a certain variable
- You will see the old and new values

Program State

Systems@ETH Zürich

- Print content of a variable:
  ```
  > (gdb) print variable|address
  > (gdb) print/x variable|address
  ```
- Treat variable as string:
  ```
  > (gdb) x/s stringvariable|address
  ```
- GDB printf:
  ```
  > (gdb) printf "%s\n", stringvariable|address
  ```
- CPU registers:
  ```
  > (gdb) info registers
  ```

Program State

Systems@ETH Zürich

You can access pointers like in C.

Pointer address:

```
> (gdb) print ptr
```

Value of struct field:

```
> (gdb) print ptr->field
```

All struct content

```
> (gdb) print *ptr
```

Useful Commands

Systems@ETH Zürich

Stack trace at seg fault

```
> (gdb) backtrace
```

Stack trace at current position (e.g. how did it get to this breakpoint)

```
> (gdb) where
```

Show source/assembly code around current position

```
> (gdb) list
> (gdb) disassemble
```

GDB UI

Systems@ETH Zürich

Layouts (GDB Native)

activate nice “TUI” layout: `> (gdb) layout asm`

default layout: `> (gdb) layout off`

Additional Programs

gdb-dashboard *(recommended)*

ddd, pwndgb, gdb-gef, …

Debug Info

Systems@ETH Zürich

Include debug info in binary:
Compile with `-g` flag and have source code available

In the assignment:
`bomb.c` has debug info

`phase_N()` does not have debug info

Bomb: Debug Info



Binary Edit

Systems@ETH Zürich

To edit your binary use a hex editor, e.g. GHex

```
> apt install ghex
> ghex <file>
```

Demo of GDB on `simple_bomb`

Assignment 5

Bomb Lab







Backstory

Systems@ETH Zürich

Welcome Mr. Powers,

Here is your individual bomb.

I am friendly enough to give you the bomb's `main` function, but it won't help much.

Setup

Systems@ETH Zürich

- Individual bomb (executable binary)
  `assignment5/bomb*/bomb`
- Bomb different from everybody else's → solution differs
- Bomb's main function given
  `assignment5/bomb*/bomb.c`

Hints

Systems@ETH Zürich

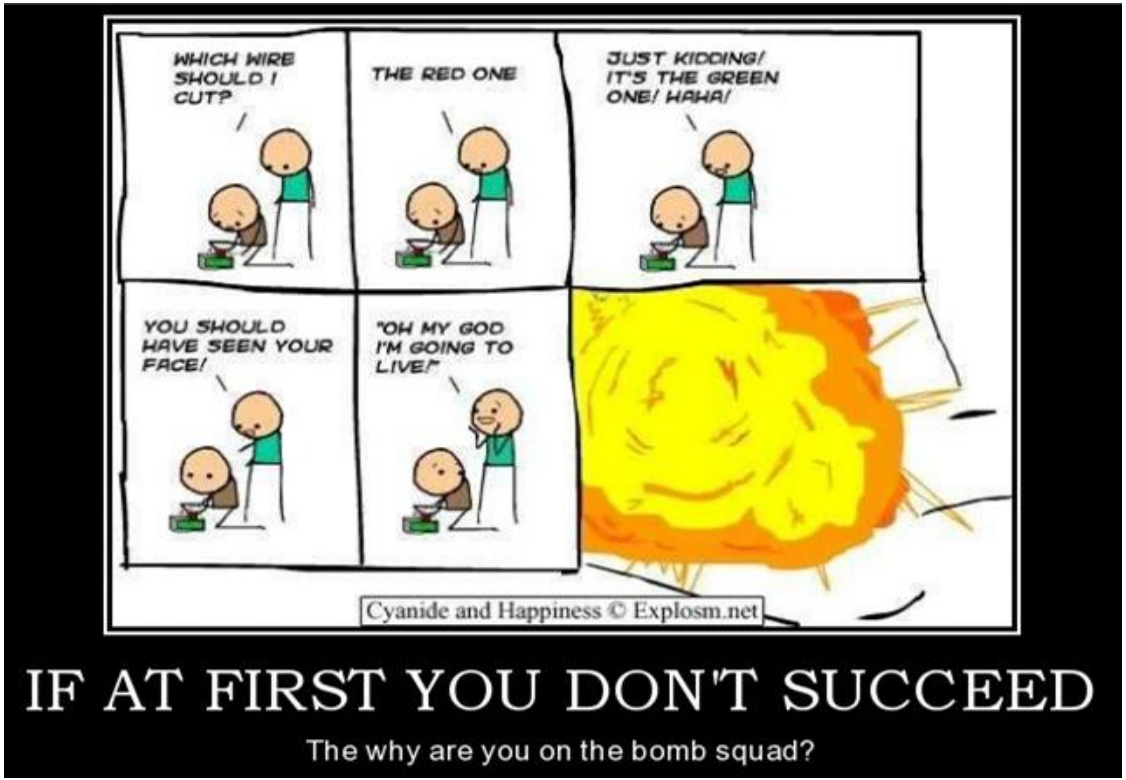
- Write key file & supply it via argument to avoid typing known keys
  `> ./bomb psol.txt`
- After using up all provided keys, bomb program switches to stdin
- Don’t go into C library functions `printf()`, `malloc()`, etc.

Strategy

Systems@ETH Zürich

- Get overview of program
- Think of when to set breakpoints (functions, lines, …) or watchpoints (variables)
- You don’t want the bomb to explode → How can you prevent that?

Strategy


WHICH WIRE SHOULD I CUT?
THE RED ONE
JUST KIDDING! IT'S THE GREEN ONE! HAHA!
YOU SHOULD HAVE SEEN YOUR FACE!
"OH MY GOD I'M GOING TO LIVE!"
Cyanide and Happiness © Explosm.net
IF AT FIRST YOU DON'T SUCCEED
The why are you on the bomb squad?

Submission



- Server graded
- Follow instructions in assignment
- Ensure path & filenames are as stated

Systems@ETH Zürich

See you next week!

THERE'S BEEN A LOT OF CONFUSION OVER 1024 vs 1000, KBYTE vs KBIT, AND THE CAPITALIZATION FOR EACH.

HERE, AT LAST, IS A SINGLE, DEFINITIVE STANDARD:

SYMBOL	NAME	SIZE	NOTES
kB	KILOBYTE	1024 BYTES OR 1000 BYTES	1000 BYTES DURING LEAP YEARS, 1024 OTHERWISE
KB	KELLY-BOOTLE STANDARD UNIT	1012 BYTES	COMPROMISE BETWEEN 1000 AND 1024 BYTES
KiB	IMAGINARY KILOBYTE	$1024\sqrt{-1}$ BYTES	USED IN QUANTUM COMPUTING
kb	INTEL KILOBYTE	1023.937528 BYTES	CALCULATED ON PENTIUM F.P.U.
Kb	DRIVEMAKER'S KILOBYTE	CURRENTLY 908 BYTES	SHRINKS BY 4 BYTES EACH YEAR FOR MARKETING REASONS
KBa	BAKER'S KILOBYTE	1152 BYTES	9 BITS TO THE BYTE SINCE YOU'RE SUCH A GOOD CUSTOMER